



MyScoutee

OFFICIAL PRODUCT DOCUMENTATION

OPERATIONS DOCUMENTATION

MyScoutee Operations Manual

Installation, configuration, availability, backup, recovery, TLS, updates, and incident response.

DOCUMENT ID MSC-OM-001	DOCUMENT VERSION 1.0.0
APPLICABLE SOFTWARE VERSION MyScoutee 1.0.0	STATUS ● Released
RELEASE DATE 2026-07-30	LANGUAGE EN

Document Control

Document status and applicability

Document owner	MyScoutee project — operations documentation
Intended audience	System operators, infrastructure owners, security officers, release managers, and incident responders
Normative deployment method	The production system installed from the <code>myscoutee_1.0.0_amd64.deb</code> package and managed by <code>systemd</code>
Review triggers	Changes to the software release, deployment topology, security boundary, backup format, or operating procedure

Version History

Version	Date	Status	Change
1.0.0	2026-07-30	Released	Initial release for operating the MyScoutee 1.0.0 Debian package.

Version Boundary

This document is based exclusively on the verified repository sources for software version 1.0.0. It may be applied to another version only after a release-specific review has verified all differences in the package, image manifest, Compose files, and operating scripts.

Evidence Base

The procedures in this manual are derived primarily from the following version-controlled sources:

- `packaging/README.md` and `README.md`;
- `packaging/compose/docker-compose.production.yml` and the replica-set extension;
- installation and removal scripts under `packaging/debian/`;
- startup, backup, restore, TLS, update, and diagnostic utilities under `packaging/scripts/`;
- unit, path, and timer files under `packaging/systemd/`;
- `server/docker/conf/nginx/` and the production Nginx entrypoint;
- `server/projects/profile/src/main/resources/application-prod.yaml`;
- `docs/plans/PROJECT_CONTEXT.md` and `docs/plans/PRODUCTION_QUALIFICATION.md`.

Table of Contents

Purpose and Scope

.....

Roles and Responsibilities

.....

Production Architecture and Data Flow

.....

Requirements and Preparation

.....

Initial Installation and Commissioning

.....

Configuration and Secret Management

.....

Service Lifecycle

.....

Health and Operational Verification

.....

Logging, Monitoring, and Capacity

.....

Backup

.....

Restore and Disaster Recovery

.....

TLS and Certificate Lifecycle

.....

Operator-Approved Updates

.....

Registry Integration and Deployment Identity

.....

Security Operations

.....

Troubleshooting and Incident Response

.....

Maintenance Checklists

.....

Removal and Reinstallation

.....

Known Operational Limitations

.....

Command and Path Quick Reference

.....

Appendix A — Source Build, Package Build, and Development Profile

.....

1. Purpose and Scope

This manual provides repeatable, verifiable, and security-conscious procedures for installing and operating a MyScoutee 1.0.0 production instance.

In Scope

- Debian/Ubuntu-style amd64 hosts that use `apt` ;
- the packaged stack running on Docker Engine and Docker Compose v2;
- Nginx, the Java API, MongoDB, Redis, MinIO, and the isolated Explore registry;
- startup, shutdown, health checks, log analysis, and capacity monitoring;
- backup and restore of MongoDB, configuration, MinIO, and the deployment registry;
- operation of TLS certificates, signed package updates, and registry connectivity;
- security, troubleshooting, and removal procedures.

Out of Scope

- installation and SQLite recovery of the separate Go registry service;
- end-user or administrator use of application features;
- vendor-specific configuration of cloud firewalls, DNS, load balancers, or external backup systems;
- non-amd64 architectures, Kubernetes, or multi-node MongoDB topologies;
- guaranteed SLA, RTO, RPO, or hardware sizing.

Source Stack and Packaged Stack

The repository file `server/docker-compose-prod.yml` defines a buildable source-production environment with relative development data paths and `pull_policy: build`. The installed package deploys `packaging/compose/docker-compose.production.yml` as `/opt/myscoutee/server/docker-compose-prod.yml`, with fixed host directories and `pull_policy: never`. When operating production, use the commands under `/opt/myscoutee` specified in this manual. The two Compose variants are not interchangeable.

2. Roles and Responsibilities

Role	Primary responsibility	Non-delegable control
Deployment owner	Business approval; definition of the domain, external providers, and recovery objectives.	Ownership of Operator access and the registry identity.
System operator	Host, Docker, systemd, backup, monitoring, TLS, capacity, and incident response.	Protection of root-level secrets, the update key, and backups.

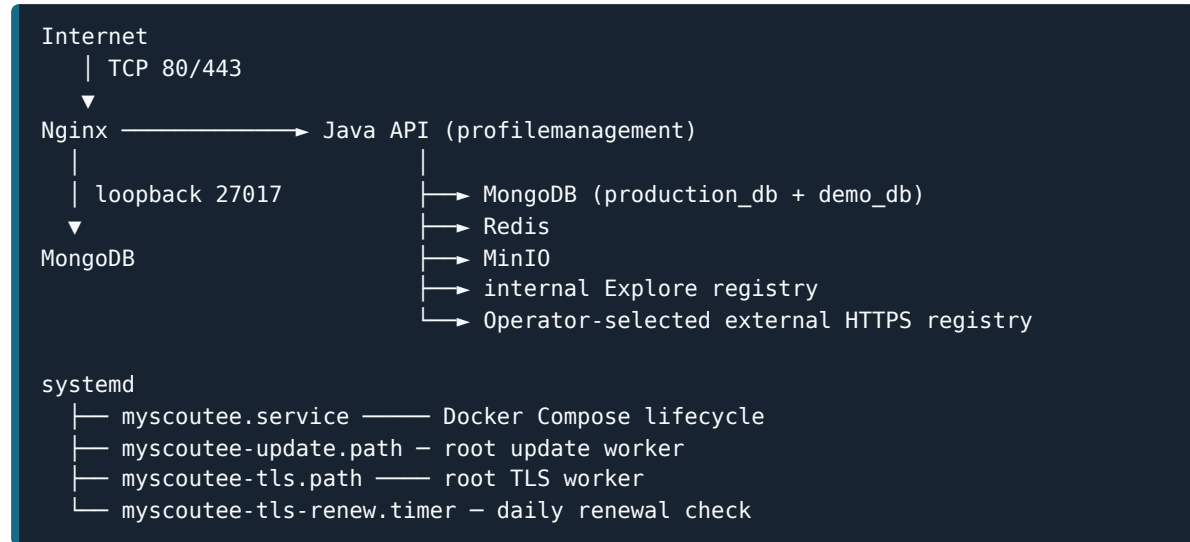
Role	Primary responsibility	Non-delegable control
Authenticated Operator	Deployment configuration, Firebase, administrators, the TLS workflow, registry catalog selection, and update approval.	Human review of the registry endpoint and scope selection and of the update manifest.
Security officer	Key and secret management, access reviews, vulnerability management, and the incident process.	Independent verification of the package-signing key fingerprint.
Release manager	Maintenance of the immutable package, image IDs, compatibility evidence, and qualification evidence.	Completion of release-specific external upgrade, restore, and security drills.

Local Decision Required

The repository does not define SLA, RTO, RPO, data-retention, or on-call targets. The deployment owner and operator must formally establish these values; backup frequency and monitoring alerts must be derived from them.

3. Production Architecture and Data Flow

Logical view of the installed production stack



Component	Container	Host access	Persistence
Reverse proxy and static frontend	nginx	0.0.0.0:80, 0.0.0.0:443; Mongo proxy only on 127.0.0.1:27017	TLS files on host mounts
Java backend	myscoutee	No direct host port; proxied by Nginx	MongoDB, registry, update, and TLS state on host mounts
MongoDB 8.0.4	mongodb	Via the Nginx stream proxy on loopback	/var/lib/myscoutee/data/m ongodb/data-v8-0
Redis 8.6.1 Alpine	redis	Internal Compose network only	No declared volume; restartable as a cache
MinIO	minio	Internal Compose network only	/var/lib/myscoutee/data/m inio
Explore demo registry	registry- demo	No host port	Two isolated Docker named volumes

Separation of Explore and the Real Registry

The Explore registry is an internal demo service with separate SQLite data, a separate signing key, and separate Java client state. It cannot be selected as the Operator's real registry and shares no state with the external registry.

4. Requirements and Preparation

Supported Installation Platform

- An `apt`-based Debian/Ubuntu system with `systemd` service management;
- amd64 architecture for the released 1.0.0 package;
- root or `sudo` privileges;
- Docker Engine and Docker Compose v2;
- a public domain and DNS when Internet-facing TLS is required;
- inbound TCP 80 and 443, while preserving SSH access in the local firewall rules;
- outbound access at minimum for missing MongoDB/Redis images, certificate issuance, and approved external providers.

Capacity Is Not Presized

The package sets no Java, MongoDB, Redis, or container memory limits, and the repository specifies no minimum CPU, RAM, or disk capacity. Host sizing must be established through release-specific load testing. The update process holds separate staging and root-cache copies and, by default, requires an additional 512 MiB reserve for each.

Preinstallation Checklist

- ☐ The supported host architecture and distribution are documented.
- ☐ Time synchronization, DNS, and the hostname work correctly.
- ☐ Disk allocation separately accounts for `/var/lib`, `/var/cache`, `/var/log`, `/var/backups`, and Docker storage.
- ☐ Ports 80 and 443 are reachable through the firewall; port 27017 is not public.
- ☐ The deployment domain and TLS mode have been approved.
- ☐ A secure method is ready for recording the shown-once Operator credential.
- ☐ Owners have been assigned for the backup target, off-host copy, retention, encryption, and restore testing.
- ☐ The installation package source and SHA-256 digest have been verified.

Optional Host Utilities

The repository provides a helper that installs only missing packages on Debian/Ubuntu systems:

```
sudo /opt/myscoutee/packaging/scripts/install-vps-prerequisites.sh
sudo /opt/myscoutee/packaging/scripts/doctor-production.sh
```

From a source checkout, run:

```
sudo bash packaging/scripts/install-vps-prerequisites.sh
bash packaging/scripts/doctor-production.sh
```

`doctor-production.sh` checks only for the presence of commands; it does not verify the running stack, disk capacity, certificates, or backups.

5. Initial Installation and Commissioning

5.1. Verify the Package

```
sha256sum myscoutee_1.0.0_amd64.deb  
dpkg-deb --info myscoutee_1.0.0_amd64.deb
```

Compare the SHA-256 value with the approved release evidence. Do not install a package of unknown origin or with a mismatched digest.

5.2. Install

```
sudo apt install ./myscoutee_1.0.0_amd64.deb
```

Do Not Use `dpkg -i` by Itself

The package declares dependencies on Docker Engine and Docker Compose v2. The documented installation path is `apt install ./...deb`, which resolves dependencies before post-installation configuration.

During the initial installation, the post-installation process:

1. creates the configuration, data, registry, update, TLS, log, and backup directories;
2. creates the Operator configuration encryption key from 32 cryptographically random bytes represented as 64 lowercase hexadecimal characters;
3. generates separate passwords for the MongoDB root user, the MongoDB application user, and MinIO;
4. when `OPERATOR_USER` is empty, creates a deployment-local Operator identifier and a random 256-bit password;
5. prints the password once, but stores only the identifier and the lowercase SHA-256 digest of the password;
6. loads the five packaged MyScoutee images, verifies the image manifest, and pulls the configured MongoDB and Redis images if they are missing;
7. enables `myscoutee.service`, the update and TLS path units, and the TLS renewal timer;
8. does not automatically start the MyScoutee application stack.

Shown Once Does Not Mean Single Use

Immediately store the deployment-local Operator password displayed during installation in a secure secret store, and protect package-manager, terminal, and automation logs. The password remains valid for subsequent local Operator sign-ins until the deployment owner disables or replaces it; enabling Firebase alone does not invalidate it.

5.3. Configure Before First Startup

```
sudoedit /etc/myscoutee/myscoutee.env
sudoedit /etc/myscoutee/myscoutee.secrets.env
sudo /opt/myscoutee/packaging/scripts/myscoutee-compose.sh config
```

Review at least the following:

- `DOMAIN` and `WEBSOCKET_ALLOWED_ORIGIN_PATTERNS` ;
- `LETSENCRYPT_ENABLED` and the selected TLS files;
- `CONTAINER_TIMEZONE` ;
- `REGISTRY_OUTBOUND_ALLOWED` , which defaults to `false` ;
- `MYSCOUTEE_UPDATE_ENABLED` , which must remain `false` without a release public key;
- Firebase, which must remain `false` until an authenticated Operator has tested and activated its configuration.

The MongoDB and MinIO password placeholders have already been replaced with genuinely random values. Do not replace them merely with a manually selected “stronger” password; rotation requires a separate database/provider procedure that this manual does not implement.

5.4. Start for the First Time

```
sudo systemctl start myscoutee
sudo systemctl status myscoutee --no-pager
sudo /opt/myscoutee/packaging/scripts/myscoutee-compose.sh ps
```

5.5. Commissioning Verification

```
curl --fail --silent --show-error http://127.0.0.1/nginx-health
sudo docker exec myscoutee wget -qO- http://127.0.0.1:8080/health
curl --fail --silent --show-error https://app.example.com/api/health
```

Replace the domain in the last command with the actual public domain. The expected backend response is `{"status": "UP"}`. External HTTPS verification is not successful until a valid public certificate is in place; using `-k` does not verify the TLS trust chain.

- ☐ The systemd unit is active.
- ☐ All six containers are running, and containers with health checks are healthy.
- ☐ HTTP application requests receive a 308 redirect to HTTPS.
- ☐ The public HTTPS certificate covers the correct domain.
- ☐ Operator sign-in succeeds.
- ☐ Deployment-specific Firebase and administrator configuration has been stored through the Operator interface or deliberately deferred.
- ☐ The first complete backup and off-host copy exist, or the limitation caused by an uninitialized registry has been documented.

6. Configuration and Secret Management

Path	Contents	Required protection
<code>/etc/myscoutee/myscoutee.env</code>	Non-secret production parameters, image references, gates, and paths	0644 at installation; modifications by root only
<code>/etc/myscoutee/myscoutee.secrets.env</code>	MongoDB/MinIO secrets, bootstrap identifier and password digest, and external provider placeholders	0600, root
<code>/etc/myscoutee/configuration/deployment-master-key</code>	At-rest encryption key for Operator-managed deployment secrets	root:10001, 0640; directory 0750
<code>/etc/myscoutee/registry/deployment-private-key.pem</code>	Deployment Ed25519 private signing key	API UID/GID 10001, 0600; directory 0700
<code>/etc/myscoutee/firebase-service-account.json</code>	Deployment-specific Firebase service account	root:10001, 0640; symbolic links are not permitted
<code>/etc/myscoutee/update/release-public-key.pem</code>	Ed25519 public verification key of the package publisher	root:10001, 0640

The Deployment Master Key and MongoDB Form One Recovery Unit

Losing or independently regenerating `deployment-master-key` makes the encrypted Operator configuration stored in MongoDB unrecoverable. Never replace the key as a “fix.” Back up and restore the key that belongs to the same database snapshot.

Controlled Configuration Change Procedure

1. Record the purpose, owner, and rollback plan for the change.
2. Create at least configuration and data backups in proportion to the risk.
3. Edit the appropriate file under `/etc/myscoutee`; do not place secrets in command-line arguments or shell history.
4. Validate Compose resolution:

```
sudo /opt/myscoutee/packaging/scripts/myscoutee-compose.sh config
```

5. Apply the change:

```
sudo systemctl restart myscoutee
```

6. Run health and functional checks, then close the change record.

Firebase, administrator, payment, email, social, and community settings managed through the authenticated Operator interface are deployment data. Saving Firebase configuration only stages it; live authentication and messaging require the real tests followed by the separate activation action.

7. Service Lifecycle

Start

```
sudo systemctl start myscoutee
```

Stop

```
sudo systemctl stop myscoutee
```

The systemd `ExecStop` runs `Compose down`. The containers and Compose network are removed, while the data directories and named volumes are retained.

Restart

```
sudo systemctl restart myscoutee
```

Automatic Startup

```
sudo systemctl enable myscoutee
sudo systemctl is-enabled myscoutee
```

Direct Compose Helper

```
sudo /opt/myscoutee/packaging/scripts/myscoutee-compose.sh ps
sudo /opt/myscoutee/packaging/scripts/myscoutee-compose.sh logs --tail=200
sudo /opt/myscoutee/packaging/scripts/myscoutee-compose.sh up -d
sudo /opt/myscoutee/packaging/scripts/myscoutee-compose.sh down
```

An Active systemd Unit Does Not Mean Every Container Is Healthy

`myscoutee.service` uses `Type=oneshot` and `RemainAfterExit=yes`. The `active (exited)` state proves that the Compose startup command ran, not that every container remains healthy. Always inspect both Compose state and container health.

Replica-Set Mode

MongoDB change-stream wakeup is optional. To enable it:

```
sudoedit /etc/myscoutee/myscoutee.env
# MONGO_REPLICA_SET_ENABLED=true
sudo systemctl restart myscoutee
```

The Compose helper then adds `docker-compose.replicaset.yml`, initializes a single `rs0` member, and retains a polling fallback. A backup and release-specific compatibility test are mandatory before switching modes.

8. Health and Operational Verification

8.1. Quick Status Check

```
sudo systemctl is-active myscoutee
sudo /opt/myscoutee/packaging/scripts/myscoutee-compose.sh ps
curl --fail --silent --show-error http://127.0.0.1/nginx-health
```

The Nginx health endpoint is exempt from the HTTP-to-HTTPS redirect and returns `UP`. It checks only the reverse-proxy process, not the complete application.

8.2. Component Checks

Component	Command	Expected result
Nginx	<code>sudo docker exec nginx nginx -t</code>	The configuration is syntactically valid and the selected key pair can be loaded.
Java API	<code>sudo docker exec myscoutee wget -q0- http://127.0.0.1:8080/health</code>	<code>{"status": "UP"}</code> ; details are not public in production.
MongoDB	<code>sudo docker inspect --format '{{.State.Health.Status}}' mongodb</code>	<code>healthy</code>
Redis	<code>sudo docker exec redis redis-cli ping</code>	<code>PONG</code>
MinIO	<code>sudo docker inspect --format '{{.State.Health.Status}}' minio</code>	<code>healthy</code> , including seed readiness.
Explore registry	<code>sudo docker exec registry-demo /registry healthcheck</code>	Success with exit code 0.

8.3. Verify the Public Endpoint

```
curl --fail --silent --show-error https://app.example.com/api/health
curl --head http://app.example.com/
curl --head https://app.example.com/
```

Expected results: the first request returns an `UP` state; a normal HTTP request is redirected to HTTPS with status 308; the HTTPS response includes the one-year HSTS header.

8.4. MongoDB 8.0 Compatibility Smoke Test

```
sudo /opt/myscoutee/packaging/scripts/smoke-mongo-compatibility.sh
```

The smoke test verifies the configured and running MongoDB version, FCV, seed data, backend health, and public content queries.

The Smoke Test Is Not Entirely Read-Only

Using the application account, the script creates, modifies, queries, and then drops a temporary `_myscoutee_mongo_compat_smoke` collection. Treat it as a planned verification activity with appropriate authorization and change logging.

8.5. Version and Image Identity

```
dpkg-query -W -f='${Package} ${Version} ${Architecture}\n' myscoutee
sudo sed -n 'lp' /opt/myscoutee/packaging/VERSION
sudo jq . /opt/myscoutee/packaging/IMAGE-MANIFEST.json
sudo docker inspect --format '{{.Name}} {{.Config.Image}} {{.Image}} {{if .State.Health}}
{{.State.Health.Status}}{{end}}' \
    nginx myscoutee minio registry-demo
```

The running image references and image IDs must match the installed manifest.

9. Logging, Monitoring, and Capacity

Service and Container Logs

```
sudo journalctl -u myscoutee.service --since "1 hour ago"
sudo /opt/myscoutee/packaging/scripts/myscoutee-compose.sh logs --since=1h --tail=500
sudo docker logs --since 1h myscoutee
sudo docker logs --since 1h nginx
sudo docker logs --since 1h mongodb
sudo docker logs --since 1h redis
sudo docker logs --since 1h minio
sudo docker logs --since 1h registry-demo
```

Privileged Worker Logs

```
sudo journalctl -u myscoutee-update.service --since "24 hours ago"
sudo journalctl -u myscoutee-tls.service --since "24 hours ago"
sudo journalctl -u myscoutee-tls-renew.service --since "30 days ago"
```

Additional persistent evidence for an update job:

```
/var/lib/myscoutee/update/runtime/jobs/<job-id>/status.json
/var/lib/myscoutee/update/runtime/jobs/<job-id>/events.ndjson
/var/lib/myscoutee/update/runtime/jobs/<job-id>/install.log
/var/log/myscoutee/update/<job-id>/request.json
```

TLS job status:

```
/var/lib/myscoutee/tls/runtime/jobs/<job-id>/status.json
/var/lib/myscoutee/tls/runtime/configuration.json
```

Capacity Checks

```
df -h /var/lib/myscoutee /var/cache/myscoutee-update /var/log/myscoutee /var/backups/myscoutee
sudo docker system df
sudo du -sh /var/lib/myscoutee/data/* /var/backups/myscoutee /var/cache/myscoutee-update 2>/dev/null
free -h
```

No Packaged Log Rotation or Alerting System

The Compose configuration does not set a logging driver or log-size limit, and the package includes no Prometheus scraper, alert manager, or disk alert. The operator must establish and test Docker daemon log rotation and host monitoring. In production, the API `/metrics` endpoint requires administrator privileges.

Minimum Alert Subjects

- any container in an `unhealthy`, `exited`, or `restart-loop` state;
- HTTP/HTTPS health failure or certificate expiration;
- capacity exhaustion in MongoDB/MinIO data, backup, update cache, or Docker storage;

- a failed or missing backup;
- `RECOVERY_REQUIRED`, a failed update, or a failed TLS job;
- registry connectivity, signature, or fingerprint errors;
- sustained growth in 5xx responses, memory use, CPU use, or restart count.

10. Backup

10.1. Routine Backup

```
sudo /opt/myscoutee/packaging/scripts/backup-production.sh
```

On an installed system, the default destination is `/var/backups/myscoutee/myscoutee-backup-YYYYMMDD-HHMMSS.tar.gz`. The archive mode is 0600.

10.2. Require a Complete Backup

```
sudo env MYSCOUTEE_BACKUP_REQUIRE_COMPLETE=true \  
/opt/myscoutee/packaging/scripts/backup-production.sh
```

The command does not produce a successful “complete” backup when any required component is missing.

10.3. Backed-Up Components

Component	Method	Notes
MongoDB application database	<code>mongodump --archive --gzip</code>	The running <code>mongodb</code> container is required.
Compose configuration	<code>.env</code> , <code>.env.secrets</code> , and <code>docker-compose-prod.yml</code>	Contains secrets.
Deployment master key	Exact file copy	Must be handled together with the MongoDB snapshot.
Real registry identity and state	Copies of the key directory and state/receipts	Only the complete key pair and state together are safe.
MinIO data	<code>tar.gz</code> of the host data directory	The script does not stop MinIO during copying.
Manifest	<code>backup-manifest.json</code>	Per-component complete/incomplete status.

10.4. Verify a Backup

```
sudo gzip -t /var/backups/myscoutee/myscoutee-backup-YYYYMMDD-HHMMSS.tar.gz  
sudo tar -x0f /var/backups/myscoutee/myscoutee-backup-YYYYMMDD-HHMMSS.tar.gz \  
./backup-manifest.json | jq .  
sudo stat -c '%a %U:%G %s %n' \  

```

```
/var/backups/myscoutee/myscoutee-backup-YYYYMMDD-HHMMSS.tar.gz  
sudo sha256sum /var/backups/myscoutee/myscoutee-backup-YYYYMMDD-HHMMSS.tar.gz
```

- ☐ The backup command returned exit code 0.
- ☐ The manifest `schemaVersion` is 1.
- ☐ `complete: true` is present when a complete backup is required.
- ☐ The archive mode is 0600.
- ☐ The SHA-256 digest was recorded separately in a protected register.
- ☐ An encrypted off-host copy exists.
- ☐ The retention and deletion policy has been applied.

The Backup Is Itself a Secret

The archive contains database credentials and, once it exists, the deployment private key. Mode 0600 on the host does not replace encryption, access logging, and integrity protection for off-host storage.

Verified Backup Limitations

- The script has no accompanying systemd timer or cron schedule.
- On an uninitialized deployment, the missing registry key/state makes the manifest incomplete; `require-complete` mode fails.
- The MinIO directory is not quiesced or stopped during backup; application consistency under write-intensive workloads must be tested separately.
- The backup excludes manual/Let's Encrypt TLS material, the Firebase service account, the update release public key, update/TLS job state, and the two Explore registry named volumes.
- There is no built-in backup encryption, remote copy, rotation, or automated restore test.

11. Restore and Disaster Recovery

Destructive Operation

The restore performs `--drop` against MongoDB and, when MinIO restore is enabled, deletes the current MinIO data directory. Before proceeding, exclude user traffic, preserve the pre-restore state, and verify that the correct backup has been selected.

11.1. Preconditions

- ☐ The selected archive's SHA-256 digest and manifest have been verified.
- ☐ The target is the same logical deployment or part of a documented clean-host disaster-recovery drill.
- ☐ Compatibility between the installed package and the backup has been verified for the specific release.
- ☐ The `mongodb` container is running.
- ☐ User traffic is isolated.
- ☐ A separate backup of the pre-restore state has been created, if still possible.
- ☐ The required `RESTORE_MINIO`, `RESTORE_CONFIG`, and `RESTORE_REGISTRY` decisions have been approved.

11.2. Basic Restore — MongoDB and Its Associated Master Key

```
sudo env CONFIRM_RESTORE=true \
/opt/myscoutee/packaging/scripts/restore-production.sh \
/var/backups/myscoutee/myscoutee-backup-YYYYMMDD-HHMMSS.tar.gz
```

The script requires both the MongoDB data and the deployment master key. It stages the key, stops the Java API, runs `mongorestore`, and activates the key only after the restore succeeds.

11.3. Extended Restore

```
sudo env CONFIRM_RESTORE=true \
  RESTORE_MINIO=true \
  RESTORE_CONFIG=true \
  RESTORE_REGISTRY=true \
/opt/myscoutee/packaging/scripts/restore-production.sh \
/var/backups/myscoutee/myscoutee-backup-YYYYMMDD-HHMMSS.tar.gz
```

Switch	Effect	Risk
<code>RESTORE_CONFIG=true</code>	Copies the saved <code>.env</code> , <code>.env.secrets</code> , and Compose file back into the server tree.	In an installed environment, verify the relationship between <code>/etc/myscoutee</code> and the server symbolic links after restore.
<code>RESTORE_MINIO=true</code>	Stops MinIO, deletes the current data directory, extracts the backup, and restarts MinIO.	The current object state is lost.

Switch	Effect	Risk
<code>RESTORE_REGISTRY=true</code>	Validates and restores the complete deployment key pair, state, and receipt history.	Transfers signing control of the registered deployment; never clone it to an independent installation.

Registry restore rejects a key-only backup, a mismatched Ed25519 key pair, an invalid fingerprint/state binding, and invalid receipt cryptography.

11.4. Post-Restore Verification

```
sudo systemctl restart myscoutee
sudo /opt/myscoutee/packaging/scripts/myscoutee-compose.sh ps
curl --fail --silent --show-error http://127.0.0.1/nginx-health
sudo docker exec myscoutee wget -qO- http://127.0.0.1:8080/health
sudo /opt/myscoutee/packaging/scripts/smoke-mongo-compatibility.sh
```

- ☐ MongoDB and the deployment master key come from the same recovery set.
- ☐ The backend, Nginx, MongoDB, Redis, MinIO, and Explore registry are healthy.
- ☐ Operator sign-in and deployment-specific provider configuration work.
- ☐ After a registry restore, the deployment fingerprint, state, and receipt history match.
- ☐ After a MinIO restore, sampled object access succeeds.
- ☐ External registry outbound access was re-enabled only after verification.

Clean-Host Disaster Recovery Is Not Generally Proven

The restore script includes strong technical checks, but the repository qualification document requires a separate clean-host restore and an upgrade drill from the oldest supported version for every release. Source tests or the successful extraction of one archive alone do not prove RTO, RPO, or complete recoverability.

Incomplete Cryptographic Restore

If replacement of the MongoDB/key pair or registry identity/state is interrupted, the script may intentionally leave the Java API stopped. Do not restart the API blindly; first establish the cause and verify the key/data pair in use.

12. TLS and Certificate Lifecycle

Production Nginx always expects TLS. It redirects normal HTTP application requests to HTTPS with status 308, except for `/nginx-health` and the ACME challenge path. HTTPS responses include a one-year HSTS header.

12.1. TLS Modes

Mode	Switch	Container path	Host storage
Operator-managed	LETSENCRYPT_ENABLED=false	/etc/nginx/keys/nginx.crt and nginx-nopass.key, or a configured absolute path	/opt/myscoutee/server/docker/conf/nginx/keys
Let's Encrypt	LETSENCRYPT_ENABLED=true	/etc/letsencrypt/live/<DOMAIN>/fullchain.pem and privkey.pem	/var/lib/myscoutee/data/certbot/conf

Post-installation creates a self-signed `localhost` pair for local inspection only. It is not a public production certificate.

12.2. Recommended Operator Workflow

1. Point public DNS to the deployment and make TCP ports 80 and 443 reachable.
2. Sign in to Configuration as an authenticated Operator.
3. Enable HTTPS management and choose automatic or manual mode.
4. Enter the public domain and, in automatic mode, the certificate contact email address.
5. Run the domain and certificate test.
6. Save the block; changing the toggle alone creates only a draft.
7. In automatic mode, enable and save automatic renewal.
8. Verify that Nginx restarts and that the external trust chain is valid.

The privileged TLS worker accepts only a fixed-schema JSON request; it does not accept arbitrary commands, paths, images, or URLs from the browser. For a manual key pair, it verifies that the certificate and key match, that the certificate covers the domain, and that the certificate remains valid for at least 24 hours.

12.3. CLI Recovery for the First Let's Encrypt Certificate

```
sudo systemctl stop myscoutee
sudo install -d -m 0755 /var/lib/myscoutee/data/certbot/conf
sudo docker run --rm --name myscoutee-certbot-bootstrap \
  -p 80:80 \
  -v /var/lib/myscoutee/data/certbot/conf:/etc/letsencrypt \
  certbot/certbot:v5.4.0 certonly --standalone \
  --non-interactive --agree-tos --no-eff-email \
  --email operator@example.com -d app.example.com
sudo sed -i \
  -e 's/^DOMAIN=.*DOMAIN=app.example.com/' \
  -e 's/^LETSENCRYPT_ENABLED=.*LETSENCRYPT_ENABLED=true/' \
  /etc/myscoutee/myscoutee.env
sudo systemctl start myscoutee
```

12.4. Manual Renewal During Recovery

```
sudo systemctl stop myscoutee
sudo docker run --rm --name myscoutee-certbot-renew \
  -p 80:80 \
  -v /var/lib/myscoutee/data/certbot/conf:/etc/letsencrypt \
```

```
certbot/certbot:v5.4.0 renew --standalone
sudo systemctl start myscoutee
```

12.5. Verify Automatic Renewal

```
sudo systemctl status myscoutee-tls-renew.timer --no-pager
sudo systemctl list-timers myscoutee-tls-renew.timer
sudo journalctl -u myscoutee-tls-renew.service --since "30 days ago"
```

The timer runs daily in persistent mode, with a randomized delay of up to two hours. It renews only when saved `AUTOMATIC` mode is enabled and `autoRenew=true`.

12.6. Verify the Certificate Externally

```
openssl s_client -connect app.example.com:443 -servername app.example.com \
  </dev/null 2>/dev/null | openssl x509 -noout -subject -issuer -dates
curl --fail --silent --show-error https://app.example.com/api/health
```

TLS Backup Gap

The production backup script saves neither the manual Nginx key pair nor the Let's Encrypt directory. These require a separate encrypted backup and reissuance procedure. `apt remove myscoutee` also removes the generated/default manual Nginx certificate and key under `/opt`.

Enabling and Provisioning Are Separate Operations

`LETSENCRYPT_ENABLED=true` does not request a certificate; it only activates an existing certificate/key pair located exactly under `DOMAIN`. Nginx startup fails when the pair is missing or invalid.

13. Operator-Approved Updates

A registry `UPDATE` announcement only reports an available release. Download and installation occur solely after explicit approval by an authenticated Operator.

13.1. Prepare the Host

1. Through an independent channel, verify the full SPKI DER SHA-256 fingerprint of the vendor's Ed25519 release public key.
2. Install the key:

```
sudo install -o root -g 10001 -m 0640 release-public-key.pem \  
/etc/myscoutee/update/release-public-key.pem
```

3. Verify the local fingerprint:

```
openssl pkey -pubin -in release-public-key.pem -outform DER | sha256sum
```

4. Configure the gate:

```
sudoedit /etc/myscoutee/myscoutee.env  
# MYSCOUTEE_UPDATE_ENABLED=true  
sudo systemctl restart myscoutee  
sudo systemctl restart myscoutee-update.path
```

5. Verify the path unit:

```
sudo systemctl status myscoutee-update.path --no-pager
```

13.2. Operator Approval

Before approval, the Operator must verify:

- ☐ the target version and release channel;
- ☐ the HTTPS artifact URL;
- ☐ the artifact size;
- ☐ the `sha256:` digest;
- ☐ the package-signing key ID;
- ☐ release compatibility and the maintenance window;
- ☐ free space for staging and cache;
- ☐ the ability to create a complete pre-update backup.

13.3. Execution Chain

1. The Java API downloads only the exact HTTPS artifact in the registry-signed manifest, subject to size and digest limits.
2. The API writes to staging, then publishes a fixed-schema request to the host inbox.
3. The systemd path unit starts a root worker, which atomically claims or quarantines the input.
4. The worker independently revalidates the package name, version, architecture, size, digest, release key ID, Ed25519 signature, image manifest, and free space.

5. It creates a mandatory production backup with `complete: true`.
6. It runs `apt-get install` from a local file.
7. It restarts the stack and, for up to 300 seconds, verifies the exact reference, ID, and health of each of the four managed running images.

Default storage limits:

- Java staging safety reserve: 512 MiB in addition to the artifact size;
- root cache minimum free reserve: 512 MiB in addition to the artifact size;
- root cache retention: two jobs;
- maximum artifact size: 4 GiB.

13.4. Monitor Progress

```
sudo journalctl -fu myscoutee-update.service
sudo cat /var/lib/myscoutee/update/runtime/jobs/<job-id>/status.json
sudo tail -f /var/lib/myscoutee/update/runtime/jobs/<job-id>/install.log
```

Terminal states are `COMPLETED`, `FAILED`, `REJECTED`, and `RECOVERY_REQUIRED`.

No Automatic Data Rollback

If installation is partial or cannot be verified, the worker sets `RECOVERY_REQUIRED`. It does not automatically restore the previous data or package. Use the backup recorded in the status, the installation log, and the release-specific recovery procedure.

13.5. Recovery

```
sudo journalctl -u myscoutee-update.service
sudo cat /var/lib/myscoutee/update/runtime/jobs/<job-id>/status.json
sudo cat /var/lib/myscoutee/update/runtime/jobs/<job-id>/install.log
sudo env CONFIRM_RESTORE=true \
  /opt/myscoutee/packaging/scripts/restore-production.sh \
  /var/backups/myscoutee/<backup-recorded-in-status>.tar.gz
```

Uninitialized Registry and the Update Gate

The backup helper treats the registry key and state as part of completeness. Before registry initialization, a backup with `MYSCOUTEE_BACKUP_REQUIRE_COMPLETE=true` fails, so the privileged update worker does not reach installation. Do not bypass this by manually creating files.

Manual `apt install ./new-package.deb` is technically possible, but it bypasses the Operator update rail's manifest, signature, complete-backup, and progress controls. Use it for a production update only as an approved, release-specific documented exception.

14. Registry Integration and Deployment Identity

14.1. Initial State

Package installation does not select a registry, create a deployment identity, register, or send telemetry. A new node is in the *installed/uninitialized* state.

Production defaults:

```
REGISTRY_ENABLED=false
REGISTRY_OUTBOUND_ALLOWED=false
REGISTRY_OPERATOR_REGISTRATION_CONFIRMED=false
REGISTRY_BASE_URL=
REGISTRY_SCOPE=
REGISTRY_CONNECT_TIMEOUT=5s
REGISTRY_REQUEST_TIMEOUT=15s
REGISTRY_RETRY_DELAY=15m
REGISTRY_MAX_RESPONSE_BYTES=1048576
REGISTRY_EXPECTED_KEY_FINGERPRINT=
REGISTRY_TRUST_ON_FIRST_USE=false
REGISTRY_ALLOW_INSECURE_HTTP=false
REGISTRY_INSECURE_HTTP_ALLOWED_HOSTS=
```

14.2. Registration Flow in the Current Registry Dialog

1. The system operator sets the host-level `REGISTRY_OUTBOUND_ALLOWED` kill switch to `true` only after outbound registry traffic has been approved.
2. The authenticated Operator opens the Registry dialog. The current field is a read-only catalog selector: the Operator selects one of the offered HTTPS endpoints and its cataloged scope.
3. The Operator initiates **Register**. The frontend sends the selected `registryBaseUrl` and `expectedRegistryScope` values to `/api/operator/registry/register`.
4. As part of the same registration action, the Java backend performs a live registry identity inspection, verifies the expected scope and any compatible existing binding, and persists the selection.
5. The backend locally creates or loads the Ed25519 deployment identity and performs registration synchronization.
6. The backend verifies the registry-signed receipt against the pinned registry public key and stores it persistently.
7. The dialog displays the lifecycle state, current URL, and success or failure of the action. Use **Disconnect** to terminate an active connection.

TOFU and Plain HTTP

Trust on first use and insecure HTTP are disabled by default in production. Do not enable them for a real registry merely for convenience. An insecure HTTP exception is only for controlled development hosts with an explicit allowlist; wildcards are not supported.

Control Gap: the Planned Two-Phase Confirmation Is Not Connected to the Current Dialog

The project context specifies a separate read-only inspection, display of the registry signing identity and fingerprint, and then explicit Operator confirmation. The backend `/inspect` and `/confirm` APIs and corresponding frontend store methods exist, but the current Registry dialog does not call them: **Register** invokes the single-phase `/register` action directly and does not display a separate fingerprint confirmation. Therefore, the current dialog must not be documented as satisfying the planned two-phase human identity control. Before enrollment with a real production registry, the release manager must treat this as an open control gap, independently approve the catalog endpoint and scope, and document either the accepted residual risk or the corrected release.

14.3. Persistent Identity and State

```
/etc/myscoutee/registry/deployment-private-key.pem  
/etc/myscoutee/registry/deployment-public-key.der  
/var/lib/myscoutee/registry/deployment-state.json  
/var/lib/myscoutee/registry/receipts/
```

The private key is PKCS#8 PEM with mode 0600; the public key is X.509 SPKI DER. The backend runs as unprivileged UID/GID 10001 but receives writable access to the registry key directory because the explicit Operator action publishes the first key pair there.

Boundary of Backend Compromise

The Java backend is the registry's only network client and possesses the deployment private key. Compromise of the backend process also endangers this key; never share the directory with another service.

14.4. Changing Registries

Changing to another registry is not a simple URL edit. The current registry must first receive a signed unregister/deactivation; only then may the old binding and receipts be archived while retaining the node key. History, claims, ledger entries, and registry identity are not migrated automatically.

14.5. Registry Operations Checklist

- ☐ The state of the host outbound kill switch is documented.
- ☐ The registry URL and scope selected from the read-only catalog have prior release- or deployment-level approval.
- ☐ The risk decision for the gap between the current single-phase dialog and the planned two-phase fingerprint confirmation is documented.
- ☐ The registry receipt verified by the backend and the deployment state are preserved in backups.
- ☐ The deployment key and its associated state/receipts are included together in the backup.

- ☐ After restore or a registry change, outbound access was enabled only with verified state.
- ☐ The internal Explore registry is not confused with the real registry.

15. Security Operations

Host and Network

- Expose only ports 80 and 443 publicly; the MongoDB host mapping must remain on loopback.
- MinIO, Redis, the Java API, and the Explore registry publish no host ports.
- Treat access to the Docker socket and membership in the `docker` group as equivalent to root privileges.
- Use a separate host-patching process for operating-system and Docker security updates; the MyScoutee package does not perform aggressive system upgrades.
- Verify that administrative access will be preserved before changing firewall rules.

Application and Container Boundaries

- The Java production image runs as UID/GID 10001 and exits when an out-of-memory condition occurs.
- The Explore registry uses a read-only root filesystem, `no-new-privileges`, and drops all capabilities.
- The TLS and update workers run as root but use fixed host paths and narrowly defined input schemas.
- For managed application images, `pull_policy: never` prevents silent image replacement behind a tag.
- The Nginx and backend upload limit is 10 MB.
- The backend does not expose detailed production health information; metrics require administrator privileges.

Secret and Key Management Rules

- ☐ Never place a private key or service account in Git, an image, a ticket, or chat.
- ☐ The release private key must never exist on an application deployment or registry server.
- ☐ Backups are encrypted, access-logged, and stored off-host.
- ☐ The deployment master key cannot be rotated without atomically re-encrypting every encrypted record.
- ☐ The registry deployment key may be restored only with a complete state/receipt recovery set.
- ☐ The shown-once Operator credential and installation log are protected.
- ☐ External provider credentials are handled through the authenticated deployment-configuration process; the package contains only empty templates.

Separate Public and Secret Data

Endpoints, fingerprints, public verification keys, Firebase web-client identifiers, and the VAPID public key are not server-authentication secrets. The Firebase service-account private key, payment/email/social tokens, MongoDB and MinIO passwords, deployment private key, and master key are secrets.

16. Troubleshooting and Incident Response

16.1. The First Five Minutes

1. Record the time, symptoms, affected users, and most recent change.
2. Do not delete containers, volumes, keys, update state, or logs.
3. Check systemd and Compose state:

```
sudo systemctl status myscoutee --no-pager
sudo /opt/myscoutee/packaging/scripts/myscoutee-compose.sh ps
```

4. Inspect the relevant container and worker logs.
5. Check free disk space, memory, and Docker storage.
6. For cryptographic or data-consistency errors, isolate traffic and stop further automated actions.

16.2. Symptom-Based Troubleshooting

Symptom	Check	Safe next step
<code>myscoutee.service</code> is active, but the site is unavailable	Compose <code>ps</code> , Nginx/backend health, container logs, listeners on 80/443, and the firewall.	Correct the cause of the unhealthy component; do not infer health from systemd's active state.
Nginx restart loop	<code>docker logs nginx</code> , <code>DOMAIN</code> , existence of the selected certificate/key, and <code>docker exec nginx nginx -t</code> if the container is running.	Restore the last known-good TLS pair and environment settings.
Backend unhealthy	MongoDB, Redis, and MinIO health; backend log; master-key path and permissions; Firebase file if enabled.	Restore the primary dependency; do not regenerate the master key.
MongoDB unhealthy or authentication error	MongoDB log, disk, <code>.env.secrets</code> , initialization data directory, and replica-set state.	Do not merely change the password in the environment file; database-user rotation requires a separate procedure.

Symptom	Check	Safe next step
MinIO unhealthy	MinIO log, data directory, permissions, free space, and seed-readiness health.	Back up the directory before deleting data; restore only from an approved archive.
Registry unavailable	Outbound kill switch, HTTPS URL/DNS/TLS, fingerprint, timeout, registry state, and backend log.	Do not enable TOFU or plain HTTP; the local application must remain available during a temporary registry failure.
Update FAILED	Job status, events, installation log, journal, release key, digest, disk, and backup manifest.	Correct the verified cause; do not recreate the same job ID.
Update RECOVERY_REQUIRED	Installed dpkg version, image manifest, running image IDs, and the backup recorded in status.	Use release-specific recovery/restore instead of automatic retry.
TLS renewal failed	DNS, public port 80, ACME challenge, timer/service journal, Certbot image, and certificate path.	Correct reachability; use the documented manual recovery command if necessary.
Disk full	<code>df</code> , Docker <code>system df</code> , backups, MongoDB/MinIO, update cache, and log size.	Delete only on the basis of evidence and the retention policy; do not run a blind <code>docker system prune</code> .

16.3. Evidence Package

For an incident, retain at least the following with timestamps and share it without secrets:

- host/distribution/kernel and Docker and Compose versions;
- installed dpkg version, image manifest, and running image IDs;
- systemd and relevant container status and logs;
- disk and memory state;
- update/TLS job status, events, and installation log;
- backup manifest and SHA-256, without the archive contents;
- the most recent configuration, TLS, registry, or release change.

Prohibited Troubleshooting “Fixes”

Do not delete or regenerate the deployment master key, registry private key/state, MongoDB data directory, MinIO data, update runtime evidence, or Docker volumes. Do not enable registry TOFU/plain HTTP, run blind purge/prune commands, or restart the API during an incomplete key/data restore.

17. Maintenance Checklists

The frequencies below are operational baselines, not built-in automation. Local SLA, RPO, and workload may require a stricter schedule.

Daily

- ☐ Check systemd and the status of all six containers.
- ☐ Check Nginx and backend health.
- ☐ Review new 5xx responses, restart loops, and update/TLS/registry errors.
- ☐ Check disk and memory thresholds.
- ☐ Verify the expected backup job's success and manifest.

Weekly

- ☐ Review growth in Docker and application logs.
- ☐ Verify the backup's off-host copy and SHA-256 digest.
- ☐ Check certificate expiration and renewal-timer state.
- ☐ Review the update cache and failed/rejected inbox and archive entries.
- ☐ Review MongoDB/MinIO storage trends and Docker image storage.

Monthly or Before a Change

- ☐ Perform a restore test in an isolated, non-production environment.
- ☐ Review Operator, administrator, root, and Docker access.
- ☐ Verify the validity of external provider credentials and connections.
- ☐ Review the registry endpoint/fingerprint and the ability to back up receipts.
- ☐ Review host OS/Docker patching, vulnerabilities, and capacity.

Before Every Release

- ☐ The candidate `.deb`, every image ID/digest, and the source commit are bound to one evidence package.
- ☐ Artifact preflight and all packaging/protocol tests have passed.
- ☐ The upgrade and interruption drill from the oldest supported version is complete.
- ☐ Clean-host restore, multi-registry, load, and security drills are complete.
- ☐ No qualification row or unaccepted security finding remains open.

```
MYSCOUTEE_QUALIFICATION_EVIDENCE_DIR=/absolute/evidence/1.2.0 \  
bash packaging/scripts/qualify-release-artifacts.sh \  
/absolute/myscoutee_1.2.0_amd64.deb \  
/absolute/myscoutee_1.0.0_amd64.deb \  
myscoutee-registry:1.2.0-prod
```

This is a read-only artifact preflight; it does not replace host-modifying upgrade and restore drills.

18. Removal and Reinstallation

Remove the Package While Retaining Data

```
sudo apt remove myscoutee
```

The process stops and removes the Compose containers and disables the application, update, and TLS units. It retains substantial state under `/etc/myscoutee` and `/var/lib/myscoutee`, the demo registry volumes, and MyScoutee image tags for a later reinstallation.

Removal of the Manual TLS Pair

The pre-removal script deletes `/opt/myscoutee/server/docker/conf/nginx/keys/nginx.crt` and `nginx-nopass.key` even during `apt remove`. When using a private CA or manual key pair, create a separate protected backup before removal.

Complete Purge

```
sudo apt purge myscoutee
```

Purge Causes Irreversible Data Loss

Purge deletes the `/etc/myscoutee`, `/opt/myscoutee`, `/var/lib/myscoutee`, `/var/log/myscoutee`, and update-cache trees; both Explore registry volumes; and every `myscoutee-api`, `myscoutee-nginx`, `myscoutee-minio`, and `myscoutee-registry` image tag. It does not delete shared upstream MongoDB/Redis images or `/var/backups/myscoutee`. A complete off-host backup and registry identity/state verification are mandatory before purge.

Pre-Removal Checklist

- ☐ Final removal has been approved.
- ☐ A complete backup, its SHA-256 digest, and an off-host copy are available.
- ☐ The separate backup gaps for TLS, Firebase, and the update key have been addressed.
- ☐ The need for a registry deactivation/exit procedure has been assessed.
- ☐ The legal basis for retaining the deployment private key and receipt history is documented.
- ☐ After purge, the backup remains reachable and readable.

19. Known Operational Limitations

Area	Limitation	Required operational treatment
Sizing	No minimum CPU, RAM, or disk requirement and no container resource limits.	Release- and deployment-specific load/capacity testing and host monitoring.
High availability	Single-host Compose and, optionally, one MongoDB replica-set member; this is not host-level HA.	Define the expected SLA accordingly; external HA requires separate design.
Redis	No declared persistent volume.	Operation must tolerate cache loss, and restart behavior must be tested.
Backup automation	No packaged timer, rotation, remote destination, or encryption.	External scheduler, retention, encryption, monitoring, and restore testing.
Backup completeness	TLS, the Firebase service account, update key/state, and Explore volumes are absent from the production backup.	A separate secret/key backup or reissuance procedure.
MinIO backup	A filesystem tar is created while the service runs, without quiescing it.	Consistency testing under write load and, if needed, a maintenance window.
Logs and alerts	No Compose log limit, scraper, or alert manager.	Host/Docker log rotation and external monitoring.
Updates	No automatic rollback; without a complete registry backup, the gate cannot be satisfied.	A qualified recovery drill and Operator-controlled restore.
Clean-host restore	It cannot be considered generally proven by source tests.	An actual clean-host/VM drill for every release candidate.
Image acquisition	The standard package contains five first-party images, but it must pull MongoDB/Redis if missing; Certbot is required for TLS operations.	Preverified outbound network access or an explicit offline image bundle for the release.
Source and package	The two production Compose variants are not identical.	On an installed system, use only the <code>/opt/myscoutee</code> wrapper and packaged Compose file.

Principle for Managing Limitations

The limitations listed here do not authorize bypassing controls. The appropriate response is a documented compensating control, a qualification drill, or a new software release.

20. Command and Path Quick Reference

Common Commands

Purpose	Command
Start	<code>sudo systemctl start myscoutee</code>
Stop	<code>sudo systemctl stop myscoutee</code>
Restart	<code>sudo systemctl restart myscoutee</code>
Stack status	<code>sudo /opt/myscoutee/packaging/scripts/myscoutee-compose.sh ps</code>
Stack logs	<code>sudo /opt/myscoutee/packaging/scripts/myscoutee-compose.sh logs --tail=200</code>
Nginx health	<code>curl -fsS http://127.0.0.1/nginx-health</code>
Backend health	<code>sudo docker exec myscoutee wget -qO- http://127.0.0.1:8080/health</code>
Complete backup	<code>sudo env MYSCOUTEE_BACKUP_REQUIRE_COMPLETE=true /opt/myscoutee/packaging/scripts/backup-production.sh</code>
Guarded restore	<code>sudo env CONFIRM_RESTORE=true /opt/myscoutee/packaging/scripts/restore-production.sh <backup.tar.gz></code>
MongoDB smoke test	<code>sudo /opt/myscoutee/packaging/scripts/smoke-mongo-compatibility.sh</code>
Update log	<code>sudo journalctl -u myscoutee-update.service</code>
TLS renewal log	<code>sudo journalctl -u myscoutee-tls-renew.service</code>

Important Host Paths

Path	Purpose
<code>/opt/myscoutee</code>	Packaged application, Compose files, scripts, and image manifest.
<code>/etc/myscoutee</code>	Deployment configuration, secrets, and keys.
<code>/var/lib/myscoutee/data</code>	MongoDB, MinIO, and Certbot data.

Path	Purpose
/var/lib/myscoutee/registry	Real registry state and receipts.
/var/lib/myscoutee/update	Update inbox, archive, and runtime state.
/var/lib/myscoutee/tls	TLS inbox, archive, configuration, and job status.
/var/cache/myscoutee-update	Root-owned update artifact cache.
/var/log/myscoutee/update	Audit evidence for accepted update requests.
/var/backups/myscoutee	Default production backup destination; retained after purge.

Emergency Sequence

1. Isolate traffic.
2. Preserve state and evidence.
3. Verify key/data pairing.
4. Perform release-specific recovery or a guarded restore.
5. Complete health, identity, and functional verification.
6. Restore traffic and registry outbound access in a controlled manner.
7. Close the incident and review the manual and procedures.

Appendix A — Source Build, Package Build, and Development Profile

This appendix covers the repository's complete lifecycle. The build and development profiles described here are not the same as the `apt`-installed production profile operated in the main body of this manual.

A.1. Build Production Artifacts from Source

From the repository root:

```
MYSCOUTEE_VERSION=1.0.0 \  
bash packaging/scripts/build-production.sh
```

By default, the script builds and synchronizes the production frontend bundle, runs the Gradle `bootJar` build, creates the base Nginx image, and then builds the source-production Compose images. The build environment must also have access to the adjacent MyScoutee registry source.

A.2. Build the Debian Package

In one step, including an automatic artifact build:

```
MYSCOUTEE_VERSION=1.0.0 \  
bash packaging/scripts/package-production.sh
```

If the preceding production build was created from exactly the same source state and version, avoid rebuilding it:

```
MYSCOUTEE_VERSION=1.0.0 \  
BUILD_ARTIFACTS=false \  
bash packaging/scripts/package-production.sh
```

Expected output: `packaging/dist/myscoutee_1.0.0_amd64.deb`. The package build saves the five managed MyScoutee images into the archive, creates an image manifest, and runs a credential audit. The resulting package may be used for production installation only after qualification and release approval.

A.3. Development Stack

```
cd server  
docker compose up --build
```

The development stack uses hot reload, workspace mounts, development/demo helpers, and services that can be inspected on loopback. The sibling `myscoutee-registry` repository is required to build the real Go registry sandbox.

The Profiles Are Not Interchangeable

The development stack, source `server/docker-compose-prod.yml`, and packaged `/opt/myscoutee/server/docker-compose-prod.yml` use different build policies, host paths, health timing, and security/operations boundaries. Development `docker compose up --build` does not replace Debian package installation, the systemd lifecycle, production TLS, guarded backup/restore, or the signed update rail.

End of document — MSC-OM-001 / 1.0.0 / Released

Applicable software version: MyScoutee 1.0.0 · Release date: 2026-07-30 · Language: EN